

Chapter 1 — Introduction to Distributed Architectures and SOA

Distributed Web Applications (AWD)

Dr. Badia Bouhdid
badiaa.bouhdid@esprit.tn

ESPRIT School of Engineering
4th-year Computer Engineering Program

Academic year 2026–2027

Learning outcomes

By the end of this chapter, students will be able to...

1

LO 1

Sections 1–2

Explain the evolution of software architectures

2

LO 2

Sections 2 and 4

Distinguish Monolith, SOA and Microservices

3

LO 3

Section 3

Identify the limits of a monolithic architecture

4

LO 4

Section 5

Understand the principles of distributed systems

5

LO 5

Section 6

Explain the role of Web services

6

LO 6

Section 7

Explain the RESTful architecture

Contents

1 Introduction

2 Evolution of Architectures

3 The Monolith and its Limits

4 Comparing Architectures

5 Distributed Systems

6 Web Services

7 RESTful Architecture

8 Summary

Section 1

Introduction

What is a software architecture?

“The software architecture of a system is the set of structures needed to reason about the system. These structures comprise software elements, relations among them, and properties of both.”

— Bass, Clements & Kazman, *Software Architecture in Practice* (2021)

Three forces push an architecture to evolve

Functional requirements

New features: mobility, real time, personalisation, multi-channel, built-in AI.

Non-functional requirements

Scaling up, 24/7 availability, security, resilience, performance.

Organisational constraints

Large and distributed teams, DevOps, frequent releases, team autonomy.

An architecture that fitted yesterday can become a bottleneck tomorrow: it must be designed to evolve.

New needs, new technical challenges

Every business expectation translates into a technical requirement on the architecture

BUSINESS NEED

Time-to-market: deliver in days, not months

Elasticity: from 100 to 10 million users

24/7 availability: SLA from 99.9% to 99.999%

Interoperability: exchange with partners (B2B)

Multi-channel: Web, mobile, IoT, chatbot

Massive data: spread across several sites

RELATED TECHNICAL CHALLENGE

→ **Continuous deployment** : without service interruption

→ **Load balancing:** across many machines

→ **Resilience:** failure becomes the norm

→ **Interface contracts:** standardised APIs

→ **Exposed business logic:** for many client types

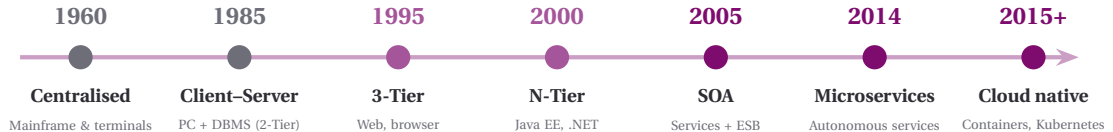
→ **Consistency & observability:** CAP theorem, tracing

Section 2

Evolution of Architectures

From the mainframe to cloud native

Each generation answers the limits of the previous one



Three underlying trends

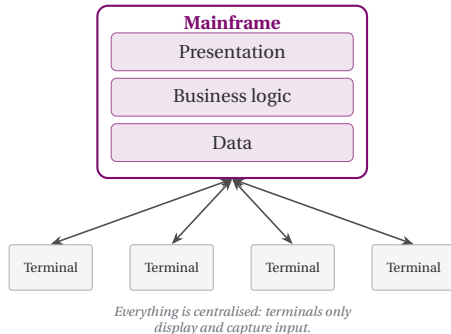
Concentration → **Distribution**

Tight coupling → **Loose coupling**

Rare deployment → **Continuous deployment**

Centralised architecture (mainframe)

1960–1980 · IBM System/360 · COBOL · dumb terminals



A single central machine runs all the processing; users access it through terminals with no computing power.

Advantages

- Centralised administration and security
- High performance, no network latency
- Strict transactional consistency (ACID)

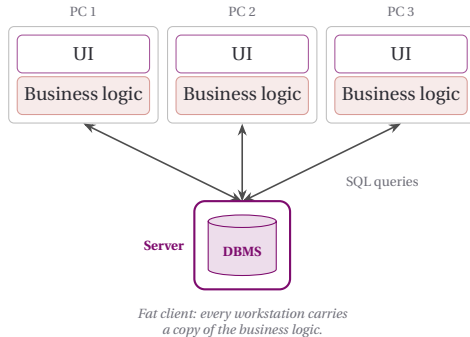
Limits

- Very high hardware cost
- Single point of failure (SPOF)
- Poor text-mode interface
- Vertical scalability only

Typical uses: banks, insurance, airline reservations — still in production today.

Client / Server (2-Tier)

1985–1995 · PC + LAN + relational DBMS · Delphi, Visual Basic



The client workstation (“fat client”) holds the interface and the business logic; a central server hosts the database.

Advantages

- Rich graphical interface (GUI)
- Load shared between clients and server
- Rapid development (RAD tools)

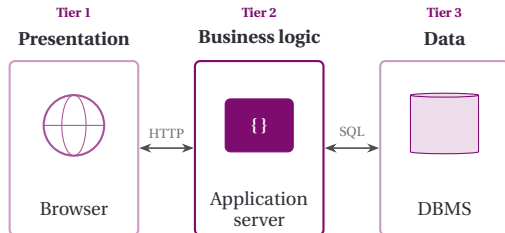
Limits

- Business logic duplicated on every PC
- Costly updates of every client (“DLL Hell”)
- Tight coupling to the database schema
- Direct database access: weak security

Typical uses: local management applications, SMEs with few users.

3-Tier architecture

From 1995 · Web (HTTP, HTML) · CGI, PHP, Servlets, ASP



Thin client: the browser only handles presentation.

The application is split into three tiers: presentation (browser), business logic (application server) and data (DBMS).

Advantages

- Clear separation of responsibilities
- Thin client: nothing to install
- Database not exposed to the client
- Each tier evolves separately

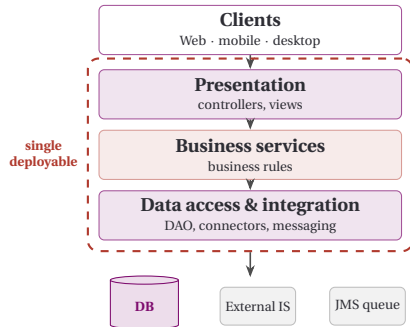
Limits

- More complex deployment
- Network latency (HTTP)
- Business tier often monolithic

Typical uses: default model for Web management applications (ERP, CRM, back offices).

N-Tier architecture

2000s · Java EE (EJB, JMS), .NET · application servers



N logical tiers... but a single deployment.

Generalisation of 3-Tier: the server side is split into specialised tiers (Web, business, data access, integration).

Advantages

- Better separation of concerns
- Scaling per tier
- Standardisation (Java EE, Spring, .NET)

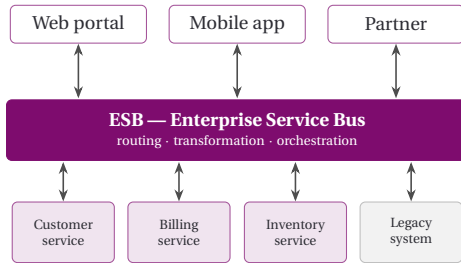
Limits

- Heavy configuration
- Cumulative latency between tiers
- Still a single deployable (.ear/.war)
- Costly distributed transactions

Typical uses: enterprise information systems: banking, insurance, telecoms, public administration.

SOA: Service-Oriented Architecture

Mid-2000s · SOAP, WSDL, UDDI · ESB · T. Erl



*SOAP / XML exchanges described by WSDL contracts.
All communication goes through the bus.*

The information system is organised into reusable business services, described by a contract and connected by an enterprise bus (ESB).

Advantages

- Reuse of services across applications
- Interoperability between technologies
- Business / IT alignment

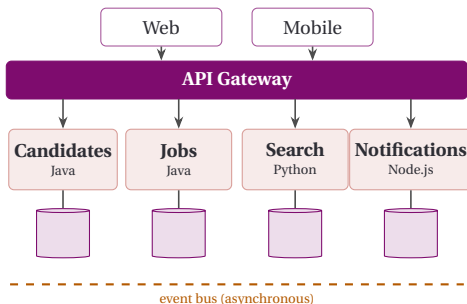
Limits

- The ESB becomes a heavy central point
- Verbose SOAP / XML
- Heavy governance and deployments
- Costly commercial platforms

Typical uses: large IT departments with heterogeneous legacy (banks, telecoms, public sector).

Microservices

2014+ · J. Lewis & M. Fowler · Netflix, Amazon, Uber



One service = one business domain, its database, its team.

The application is a set of small autonomous services, each focused on a business domain, with its own database and its own deployment.

Advantages

- Independent and frequent deployment
- Fine-grained scalability, service by service
- Fault isolation
- Technology freedom (polyglot)

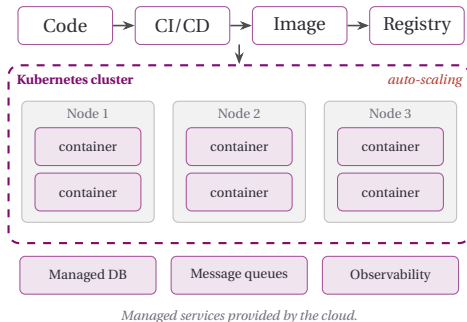
Limits

- High operational complexity
- Hard distributed data consistency
- Entry cost (CI/CD, monitoring)
- Unsuitable for small projects

Typical uses: fast-growing platforms: e-commerce, streaming, SaaS.

Cloud native

2015+ · CNCF · containers, Kubernetes, CI/CD, managed services



Applications designed for the cloud: containerised services, automatically orchestrated, continuously delivered and observable.

Advantages

- Automatic elasticity, pay-as-you-go
- Multi-zone resilience
- Continuous delivery
- Managed services (DB, queues, AI)

Limits

- Vendor lock-in
- Hard-to-predict costs
- Specialised skills (SRE, FinOps)
- Data sovereignty and compliance

Typical uses: SaaS, fast-growing startups, IoT, data platforms.

Section 3

The Monolith and its Limits

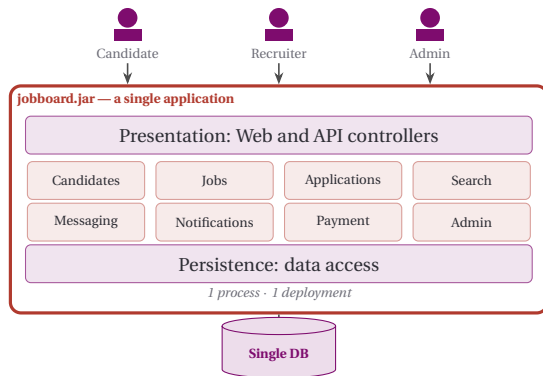
What is a monolith?

Running example: JobBoard, an online recruitment platform

Definition

A **monolithic application** bundles all its features into a **single deployable unit**: one process, one code base, one database, one release cycle.

- **1 deployable artifact**: a single .jar / .war to ship
- **1 process**: modules call each other in memory
- **1 database**: shared by all modules
- **1 repository, 1 cycle**: all teams release together



Strengths and weaknesses of the monolith

Strengths

- **Simple development:** one project, one build, one debugger
- **Simple deployment:** a single artifact to ship
- **Performance:** in-memory calls, no network
- **ACID transactions:** one database, immediate consistency
- **Easy end-to-end testing:** everything starts locally
- **Low initial cost:** little infrastructure to manage

Weaknesses

- **Creeping coupling:** towards a “big ball of mud”
- **All-or-nothing deployment:** one change, redeploy everything
- **Coarse scalability:** the whole application is duplicated
- **Fragility:** a local failure can stop everything
- **Technology lock-in:** one language, one framework
- **Hard collaboration:** conflicts between teams

The monolith remains an excellent starting point: its limits appear with growth.

The 7 structural limits of the monolith

Overview before going into detail

#1 Maintenance

Code base too large to be mastered

#2 Coupling

Entangled modules, cascading side effects

#3 Deployment

Rebuild and redeploy everything for one change

#4 Scalability

Duplicate the whole application for one saturated module

#5 Availability

A local failure stops the whole application

#6 Collaboration

Teams block each other

#7 Technical debt

Frozen language and framework

These limits emerge gradually, as code, teams and traffic grow.

Maintenance, coupling, deployment

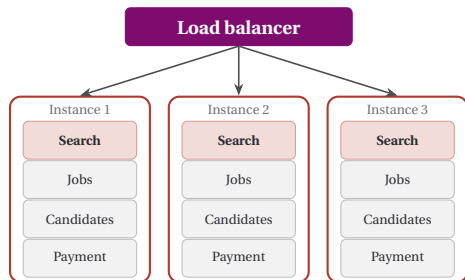
Limits #1 to #3

Limit	Cause	In JobBoard
#1 Maintenance	Beyond a few hundred thousand lines, nobody masters the whole code base any more. Maintenance accounts for the largest share of a system's lifetime cost (Sommerville).	Fixing the subscription fee calculation requires understanding four modules.
#2 Coupling	Without discipline, modules call each other directly and share the same tables: the code becomes a “big ball of mud” (Foote & Yoder, 1997).	Changing the Candidate table breaks Search and Messaging.
#3 Deployment	Every change, even a minor one, requires rebuilding, retesting and redeploying the whole application: releases become rare and risky.	Fixing a notification label also forces a redeployment of Payment.

Limit #4: scalability

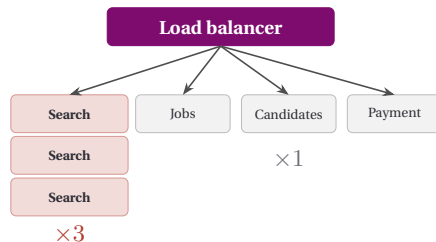
Duplicate the whole application... or only what is saturated

Monolith: everything is duplicated



3 copies of everything, even rarely used modules

Split: only what is saturated is duplicated



Resources follow the actual load of each function

Vertical scaling = a more powerful machine (limited, costly) (elastic) · **Horizontal scaling** = more machines

Availability, collaboration, technical debt

Limits #5 to #7

Limit	Cause	In JobBoard
#5 Availability	All modules share the same process and memory: a memory leak in one module brings down the whole application (cascading failure; Nygard, <i>Release It!</i>).	A memory leak in Notifications makes Payment unavailable.
#6 Collaboration	Conway's law (1968): a system mirrors the structure of the organisation that builds it. Several teams on one code base = conflicts, waiting, a shared release.	The Payment and Search teams wait for the same release.
#7 Technical debt	Shortcuts pile up (W. Cunningham, 1992). Upgrading the language or framework becomes too risky: the technology stack freezes.	Impossible to integrate a recommendation engine written in Python.

Section 4

Comparing Architectures

Four styles side by side

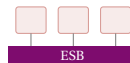
Three criteria are enough to tell them apart



Monolith



N-Tier



SOA



Microservices

Deployment unit

1 artifact

1 artifact, N layers

Enterprise services

1 artifact per service

Communication

In-memory calls

Calls between layers

Through the ESB
(SOAP/XML)

Lightweight APIs
(REST) + events

Data

1 shared database

1 shared database

Often shared

1 database per
service

Comparison table

Summary of the selection criteria

Criterion	Monolith	N-Tier	SOA	Microservices
Coupling	Tight (implicit)	Medium	Loose	Very loose
Deployment unit	1 artifact	1 artifact	Per service	Per service
Scalability	Global	Per tier	Per service	Fine-grained, per service
Data	1 database	1 database	Often shared	1 database per service
Communication	In memory	Between layers	ESB (SOAP)	REST / events
Reuse	Low	Medium	Very high	High
Operational complexity	Low	Medium	High	Very high
Speed of change at scale	Low	Moderate	Low	High
Initial cost	Low	Medium	High	High

Which architecture, when?

Context matters more than fashion

Context		Recommended style	Why
Startup, product to validate	→	Modular monolith	<i>simplicity, time-to-market</i>
SME, stable internal system	→	N-Tier	<i>proven, controlled cost</i>
Large IT department, heterogeneous legacy	→	SOA	<i>integration, reuse</i>
Fast-growing SaaS platform	→	Microservices	<i>scalability, team autonomy</i>
IoT, streaming, big data	→	Cloud native + events	<i>elasticity, large volumes</i>
Embedded / edge	→	Compact monolith	<i>hardware constraints</i>

There is no “best” architecture: only architectures suited to a context.

Section 5

Distributed Systems

What is a distributed system?

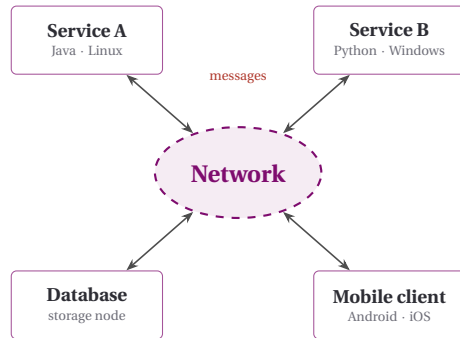
“A distributed system is one in which components located at networked computers communicate and coordinate their actions only by passing messages.”

— Coulouris, Dollimore, Kindberg & Blair (2011)

“A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.”

— Leslie Lamport (1987)

Examples: the Web, DNS, video streaming, a microservices application.



No shared memory, no common clock.

Fundamental characteristics

What makes a distributed system different from a local application

Concurrency

Components run in parallel on several machines and share resources.

No global clock

Each machine has its own clock: the order of events must be reconstructed (Lamport's logical clocks, 1978).

Independent failures

A node can fail without the others knowing immediately: failure is *partial*.

Heterogeneity

Different hardware, operating systems, languages and networks must cooperate.

Openness

The system is extended through published interfaces and open standards.

Scalability

The system must absorb growth in users and data without changing its design.

Consequence: the network becomes part of the program — latency, message loss and partial failures must be designed for, not ignored.

Transparency: hiding distribution

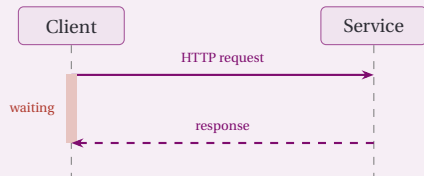
Goal: the whole should appear to the user as a single coherent system (ISO RM-ODP)

Transparency	What is hidden	Example
Access	differences between local and remote access	calling a remote service like a local method
Location	the physical location of the resource	a URL or DNS name rather than an IP address
Migration	the movement of a resource	a container restarted on another node
Replication	the existence of several copies	several instances behind a load balancer
Concurrency	sharing between users	simultaneous bookings without conflict
Failure	failures and their recovery	automatic retry, failover
Performance	reconfiguration according to load	automatic addition of instances
Scaling	the growth of the system	adding servers without changing the application

Communicating: synchronous or asynchronous?

Two main ways of exchanging messages between components

Synchronous (request / response)

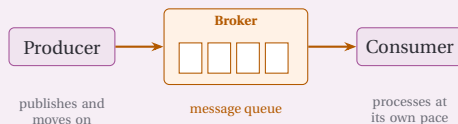


Examples: HTTP / REST, gRPC

+ simple, immediate result

– temporal coupling: if the service is down, the caller fails

Asynchronous (messages)



Examples: RabbitMQ, Apache Kafka

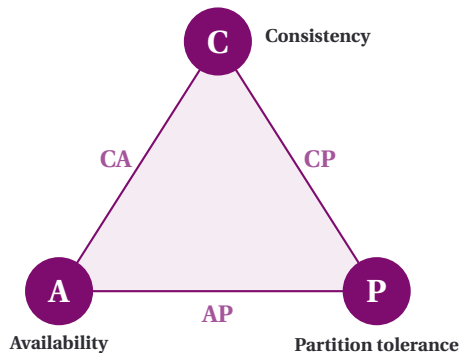
+ decoupling, absorbs load peaks

– eventual consistency, harder debugging

Rule of thumb: synchronous when the caller needs the answer *now* (read a job offer); asynchronous for notifications and long-running processing (send confirmation e-mails).

The CAP theorem

E. Brewer (2000), proven by S. Gilbert & N. Lynch (2002)



- **C : Consistency:** all nodes see the same data at the same time
- **A : Availability:** every request receives a response
- **P : Partition tolerance:** the system keeps working despite a network split

During a partition, you must choose between C and A.

CP : refuse rather than answer wrongly. *E.g. a bank balance (ZooKeeper, HBase).*

AP : answer, even if slightly out of date. *E.g. a “like” counter (Cassandra, DynamoDB).*

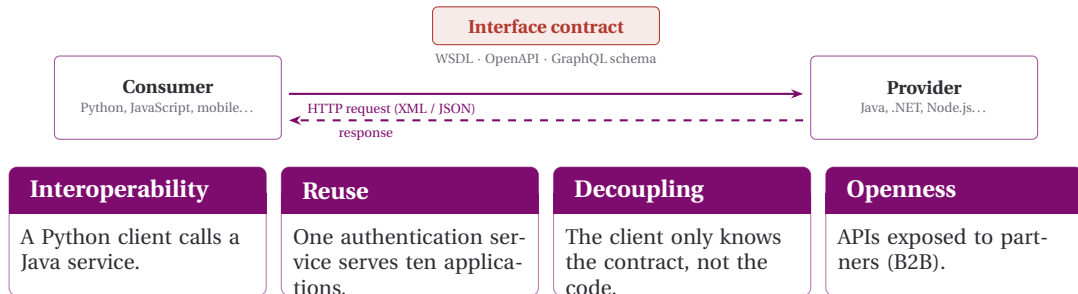
Section 6

Web Services

What is a Web service?

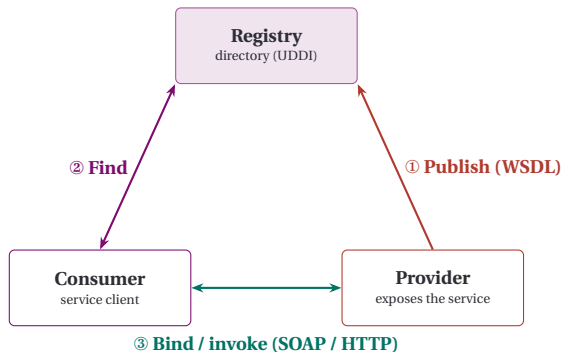
“A Web service is a software system designed to support interoperable machine-to-machine interaction over a network. It has an interface described in a machine-processable format.”

— W3C, *Web Services Architecture* (2004)



The publish – find – bind model

The three roles of the Web services architecture (W3C)



Related standards

- SOAP** format of the exchanged messages (XML envelope)
- WSDL** contract description: operations, parameters, address
- UDDI** registry to publish and look up services

Today

The registry has been replaced by API portals (OpenAPI) and by **service discovery** (e.g. Eureka) in microservices.

SOAP, REST, GraphQL: three families

Overview — REST is detailed in Section 7 and in Chapter 2

SOAP

1998–2000 · W3C

XML messaging **protocol**, WSDL contract.

Strengths

- Strict, well-tooled contract
- Security & transactions (WS-*)

Limits

- Verbose (XML)
- Heavy to implement

REST

2000 · R. Fielding

Architectural **style** over HTTP: resources + verbs, usually JSON.

Strengths

- Simple and universal (HTTP)
- Lightweight, uses HTTP caching

Limits

- Optional contract (OpenAPI)
- Over- or under-fetching

GraphQL

2015 · Facebook

Typed **query language**, a single endpoint.

Strengths

- The client picks the fields
- Strongly typed schema

Limits

- More complex HTTP caching
- Costly queries to control

Section 7

RESTful Architecture

REST: everything is a resource

Representational State Transfer : Roy Fielding, PhD dissertation (2000)

- **Resource**: any identifiable business entity: a job, a candidate
- **URI**: unique identifier: `/api/jobs/42`
- **Representation**: the state of the resource that is transferred, usually JSON
- **HTTP verb**: action on the resource: GET, POST, PUT, DELETE

REST is neither a protocol nor a standard: it is an **architectural style** defined by constraints.

Client request

```
GET /api/jobs/42 HTTP/1.1
Accept: application/json
```

Server response

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "id": 42,
  "title": "Java Developer",
  "city": "Tunis",
  "available": true }
```

The six constraints of a REST architecture

An API that follows these constraints is called RESTful

1. Client-server

Separate the user interface from data and logic: each side evolves independently.

2. Stateless

Each request carries all the information needed: the server keeps **no session**.

3. Cacheable

Responses state whether they can be cached (HTTP headers).

4. Uniform interface

Resources identified by URIs, manipulated through representations, self-descriptive messages.

5. Layered system

The client does not know whether it talks to the end server or to an intermediary (gateway, cache, proxy).

6. Code on demand

Optional. The server can send executable code to the client (e.g. JavaScript).

Why stateless matters: any server instance can answer any request, so instances can be added behind a load balancer.

Example: the JobBoard REST API

One resource, HTTP verbs, status codes

Verb	URI	Action	Typical response
GET	/api/jobs	List the jobs	200 OK
GET	/api/jobs/42	Read one job	200 OK / 404 Not Found
POST	/api/jobs	Create a job	201 Created
PUT	/api/jobs/42	Replace a job	200 OK
DELETE	/api/jobs/42	Delete a job	204 No Content
GET	/api/jobs/42/applications	Applications for a job	200 OK

2xx Success
200, 201, 204

3xx Redirection
301, 304

4xx Client error
400, 401, 404, 409

5xx Server error
500, 503

Good practice: plural nouns in the URI, never a verb (avoid /getJob) — the action is carried by the HTTP method.

Section 8

Summary

Key takeaways

One key idea per learning outcome

LO 1 — Evolution

Architectures evolve under the pressure of business needs, load and organisation.

LO 2 — Styles

Styles differ by deployment unit, communication mode and data management.

LO 3 — Monolith

Ideal to get started; growth reveals its limits (deployment, scalability, teams).

LO 4 — Distribution

Machines coordinate through messages: partial failure and latency are the norm.

LO 5 — Web services

Components become interoperable through contracts and open standards.

LO 6 — REST

Resources identified by URIs, manipulated through HTTP verbs, stateless server.

A good architecture is not the newest one: it is the one that fits the present and future context.

Test your knowledge

One question per learning outcome

- LO 1** Which limit of the Client / Server model does the 3-Tier architecture solve?
- LO 2** What distinguishes SOA from microservices in terms of communication and data?
- LO 3** Why does a monolith force you to “duplicate everything” to scale up?
- LO 4** During a network partition, why must a distributed system choose between consistency and availability? Give an example of each choice.
- LO 5** What are the three roles of the publish – find – bind model?
- LO 6** Why does the “stateless” constraint make a REST API easier to scale?

References

- [1] L. Bass, P. Clements, R. Kazman. *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
- [2] G. Coulouris, J. Dollimore, T. Kindberg, G. Blair. *Distributed Systems: Concepts and Design*, 5th ed. Pearson, 2011.
- [3] A. S. Tanenbaum, M. van Steen. *Distributed Systems*, 3rd ed., 2017.
- [4] T. Erl. *Service-Oriented Architecture: Concepts, Technology, and Design*. Prentice Hall, 2005.
- [5] M. Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [6] S. Newman. *Building Microservices*, 2nd ed. O'Reilly, 2021.
- [7] C. Richardson. *Microservices Patterns*. Manning, 2018.
- [8] M. Nygard. *Release It!*, 2nd ed. Pragmatic Bookshelf, 2018.
- [9] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, UC Irvine, 2000.
- [10] J. Lewis, M. Fowler. *Microservices*. martinowler.com, 2014.
- [11] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 1978.
- [12] S. Gilbert, N. Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 2002.
- [13] M. E. Conway. How do committees invent? *Datamation*, 1968.
- [14] B. Foote, J. Yoder. Big Ball of Mud. *PLoP*, 1997.
- [15] W3C. *Web Services Architecture*. W3C Working Group Note, 2004.

Thank you!

Questions? · Next chapter: REST APIs